

MEDIÇÕES DO SLA

Nome do Serviço 1: Autenticação e Registro de Vulnerabilidades

Este serviço engloba o fluxo crítico de segurança: autenticação do usuário via token e o registro de uma nova vulnerabilidade no sistema.

- **Tipo de operações:** Leitura (Login) e Inserção (Cadastro).
- **Arquivos envolvidos:**
 - src/main/java/com/example/backend_vuln/controller/AuthController.java
 - src/main/java/com/example/backend_vuln/service/AuthService.java
 - src/main/java/com/example/backend_vuln/controller/VulnerabilidadeController.java
 - src/main/java/com/example/backend_vuln/config/SecurityConfig.java
- **Arquivos com o código fonte de medição do SLA:**
 - test_carga.js (Script k6)
- **Data da medição:** 22/11/2025
- **Descrição das configurações:**
 - **Ambiente:** Execução Local (Windows).
 - **Runtime:** Java 21 (Spring Boot 3).
 - **Banco de Dados:** H2 Database (In-Memory) configurado para eliminar latência de disco e isolar o desempenho da aplicação.
 - **Segurança:** Spring Security com algoritmo BCrypt.

Testes de carga (SLA):

- **Latência (Tempo de resposta médio):** 49.14 ms (com 10 usuários) e 225.75 ms (com 100 usuários)
 - *Pior caso (p95):* 101.25 ms (com 10 usuários) e 567.9 ms (com 100 usuários)
- **Vazão (Throughput):** 14.64 requisições/segundo (com 10 usuários) e 109.17 requisições/segundo (com 100 usuários)
- **Concorrência:** 10/100 VUs (Usuários Virtuais simultâneos)

LEVANTAMENTO DE HIPÓTESES dos potenciais gargalos:

Com base na comparação dos testes de carga realizados com 10 VUs (latência média de **49ms**) e 100 VUs (latência média de **225ms**), observou-se uma degradação de performance de aproximadamente **4,5 vezes** ao aumentar a concorrência em 10 vezes. A principal hipótese é:

1. Saturação de CPU devido à Criptografia (CPU Bound) A hipótese primária é que o algoritmo de hash de senha utilizado (**BCrypt**) é o principal gargalo. O BCrypt é intencionalmente projetado para ser lento e consumir ciclos de processamento (CPU) para desencorajar ataques de força bruta.

- **Evidência:** Com 10 usuários, a CPU do servidor consegue processar os hashes quase simultaneamente. Com 100 usuários concorrentes solicitando login/cadastro, a CPU atinge 100% de utilização, criando uma fila de processamento (thread queuing), o que eleva drasticamente o tempo de resposta (p95 atingindo 567ms).

```
PS C:\Users\jvpma\Downloads\risk-security-platform-feature-implementacao-auth> k6 run test_carga.js

Grafana
K6

execution: local
script: test_carga.js
output: -

scenarios: (100.00%) 1 scenario, 10 max VUs, 2m10s max duration (incl. graceful stop):
  * default: Up to 10 looping VUs for 1m40s over 3 stages (gracefulRampDown: 30s, gracefulStop: 30s)

THRESHOLDS

http_req_duration
✓ 'p(95)<2000' p(95)=101.25ms

http_req_failed
✓ 'rate<0.05' rate=0.00%

TOTAL RESULTS

checks_total ..... 1472 14.645502/s
checks_succeeded... 100.00% 1472 out of 1472
checks_failed ..... 0.00% 0 out of 1472

✓ login realizado
✓ vulnerabilidade criada
```

```
HTTP
http_req_duration.....: avg=49.14ms min=3.62ms med=71.95ms max=511.29ms p(90)=96.6ms p(95)=101.25ms
{ expected_response:true }....: avg=49.14ms min=3.62ms med=71.95ms max=511.29ms p(90)=96.6ms p(95)=101.25ms
http_req_failed.....: 0.00% 0 out of 1472
http_reqs.....: 1472 14.645502/s

EXECUTION
iteration_duration.....: avg=1.09s min=1.07s med=1.09s max=1.65s p(90)=1.11s p(95)=1.11s
iterations .....: 736 7.322751/s
VUS.....: 1 min=1 max=10
VUS_max.....: 10 min=10 max=10

NETWORK
data_received.....: 134 MB 1.3 MB/s
data_sent.....: 404 kB 4.0 kB/s
```

```
PS C:\Users\jvpma\Downloads\risk-security-platform-feature-implementacao-auth> k6 run test_carga.js

Grafana
K6

execution: local
script: test_carga.js
output: -

scenarios: (100.00%) 1 scenario, 100 max VUs, 2m10s max duration (incl. graceful stop):
  * default: Up to 100 looping VUs for 1m40s over 3 stages (gracefulRampDown: 30s, gracefulStop: 30s)

THRESHOLDS

http_req_duration
✓ 'p(95)<5000' p(95)=567.9ms

http_req_failed
✓ 'rate<0.05' rate=0.00%

TOTAL RESULTS

checks_total ..... 10994 109.170957/s
checks_succeeded... 100.00% 10994 out of 10994
checks_failed ..... 0.00% 0 out of 10994

✓ login realizado
✓ vulnerabilidade criada
```

```

HTTP
http_req_duration.....: avg=225.75ms min=2.64ms med=193.55ms max=1.48s p(90)=467.4ms p(95)=567.9ms
  { expected_response:true }...: avg=225.75ms min=2.64ms med=193.55ms max=1.48s p(90)=467.4ms p(95)=567.9ms
http_req_failed.....: 0.00% 0 out of 10994
http_reqs.....: 10994 109.170957/s

EXECUTION
iteration_duration.....: avg=1.46s min=1.07s med=1.43s max=2.82s p(90)=1.84s p(95)=1.97s
iterations.....: 5497 54.585479/s
vus.....: 7 min=4 max=100
vus_max.....: 100 min=100 max=100

NETWORK
data_received.....: 1.0 GB 10 MB/s
data_sent.....: 3.0 MB 30 kB/s

```

CÓDIGO DE TESTE DE ESCRITA

***Obs:** Para efeitos de comparação entre 10 e 100 usuários, mudamos o target de 10 para 100, e também aumentamos o http_req_duration para 5000, a fim de aumentar o tempo limite.

```

import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
  stages: [
    { duration: '30s', target: 10 }, // Sobe para 10 usuários
    { duration: '1m', target: 10 }, // Mantém 10 usuários por 1
minuto
    { duration: '10s', target: 0 }, // Desce para 0
  ],
  thresholds: {
    http_req_duration: ['p(95)<2000'], // 95% das requisições
abaixo de 2s
    http_req_failed: ['rate<0.05'], // Menos de 5% de erro
  },
};

const BASE_URL = 'http://127.0.0.1:8080/api';

export default function () {
  // LOGIN (Teste de Leitura e CPU)
  const payloadLogin = JSON.stringify({
    email: 'teste@email.com', // Garanta que esse user existe ou
use um válido
    senha: '123',
  });

  const params = {
    headers: { 'Content-Type': 'application/json' },
  };

```

```

    let resLogin = http.post(`${BASE_URL}/auth/login`, payloadLogin,
params);

    const loginSucesso = check(resLogin, { 'login realizado': (r) =>
r.status === 200 });

    // CADASTRO (Teste de Escrita) - Só faz se logou
    if (loginSucesso) {
        const randomId = Math.floor(Math.random() * 100000);

        const payloadVuln = JSON.stringify({
            titulo: `Vuln Carga ${randomId}`,
            descricao: "Teste via k6 no IntelliJ",
            sistema_impactado: "Servidor",
            criticidade: "ALTA",
            cve: "CVE-TESTE",
            metricasCVSS: { vectorString:
"CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H" }
        });

        let resCadastro =
http.post(`${BASE_URL}/vulnerabilidades/usuario/1`, payloadVuln,
params);

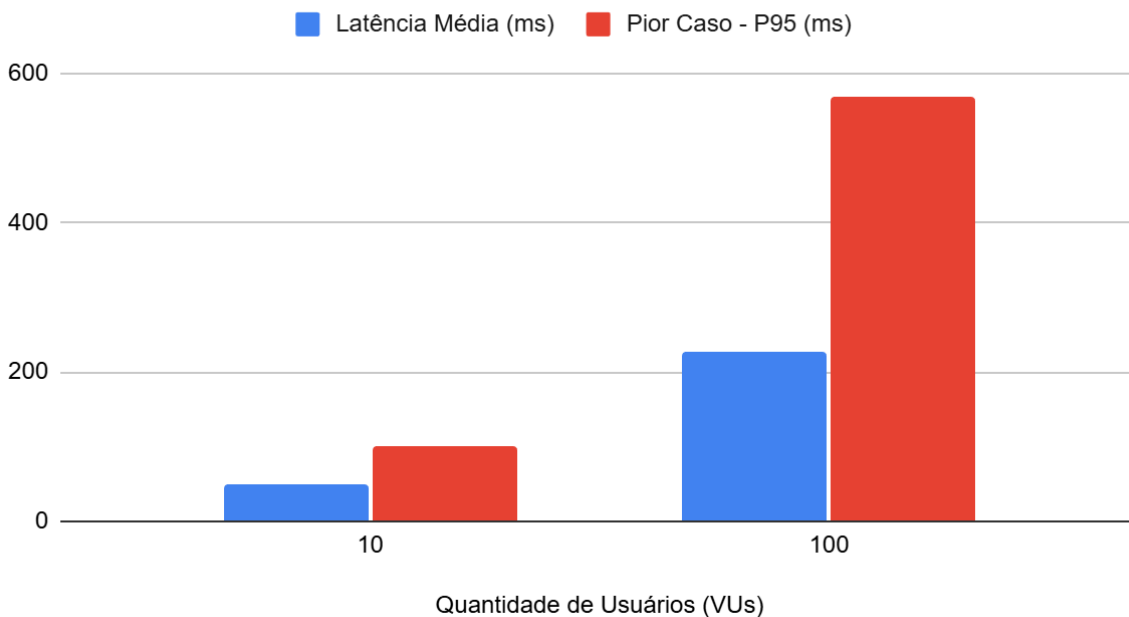
        check(resCadastro, { 'vulnerabilidade criada': (r) => r.status
=== 200 || r.status === 201 });
    }

    sleep(1);
}

```

COMPARAÇÃO DE LATÊNCIA DE ESCRITA E PIOR CASO ENTRE TESTES COM 10 E 100 VUS

Latência Média (ms) e Pior Caso - P95 (ms)



Nome do Serviço 2: Listagem de Vulnerabilidades

Este serviço representa a consulta de dados públicos ou privados do sistema, uma operação típica de leitura.

- **Tipo de operações:** Leitura (GET).
- **Arquivos envolvidos:**
 - `src/main/java/com/example/backend_vuln/controller/VulnerabilidadeController.java`
 - `src/main/java/com/example/backend_vuln/repository/VulnerabilidadeRepository.java`
- **Arquivos com o código fonte de medição do SLA:**
 - `test_leitura.js` (Script k6)
- **Data da medição:** 22/11/2025
- **Descrição das configurações:**
 - **Ambiente:** Execução Local (Windows).
 - **Runtime:** Java 21 (Spring Boot 3).
 - **Banco de Dados:** H2 Database (In-Memory).

Testes de carga (SLA):

- **Latência (Tempo de resposta médio):** 4.33 ms (com 20 usuários) e 2.08 ms (com 100 usuários)
 - *Pior caso (p95):* 6.32 ms (com 20 usuários) e 5.13 ms (com 100 usuários)
- **Vazão (Throughput):** 16.63 requisições/segundo (com 20 usuários) e 50.62 requisições/segundo (com 100 usuários)

- **Concorrência:** 20/100 VUs (Usuários Virtuais simultâneos)

LEVANTAMENTO DE HIPÓTESES dos potenciais gargalos:

No teste de estresse com 100 VUs, o serviço de listagem apresentou desempenho excepcional (latência média de ~2ms e vazão de ~50 reqs/s). Diferente do cenário de escrita, o processamento não é o limitador. Para este cenário, os potenciais gargalos em escalas maiores seriam:

1. Largura de Banda e Serialização (Network/CPU Overhead) Embora a consulta ao banco em memória seja rápida, transformar uma lista grande de objetos Java em JSON e trafegar esses dados pela rede consome recursos.

- **Hipótese:** Se o banco de dados crescer (ex: 10.000 vulnerabilidades cadastradas), o *payload* de resposta ficará grande. Com 100 usuários baixando esse volume simultaneamente, o gargalo passará a ser a **largura de banda da rede** ou o tempo de CPU gasto apenas para montar o JSON gigante.

```
PS C:\Users\jvpma\Downloads\risk-security-platform-feature-implementacao-auth> K6 run test_leitura.js

Grafana

execution: local
script: test_leitura.js
output: -

scenarios: (100.00%) 1 scenario, 20 max VUs, 1m30s max duration (incl. graceful stop):
  * default: Up to 20 looping VUs for 1m0s over 3 stages (gracefulRampDown: 30s, gracefulStop: 30s)

THRESHOLDS

http_req_duration
  ✓ 'p(95)<500' p(95)=6.32ms

http_req_failed
  ✓ 'rate<0.01' rate=0.00%

TOTAL RESULTS

checks_total..... 1010 16.639599/s
checks_succeeded... 100.00% 1010 out of 1010
checks_failed..... 0.00% 0 out of 1010

✓ listagem realizada
```

```
HTTP
http_req_duration..... avg=4.33ms min=1.55ms med=3.57ms max=441.09ms p(90)=5.4ms p(95)=6.32ms
  { expected_response:true }... avg=4.33ms min=1.55ms med=3.57ms max=441.09ms p(90)=5.4ms p(95)=6.32ms
http_req_failed..... 0.00% 0 out of 1010
http_reqs..... 1010 16.639599/s

EXECUTION
iteration_duration..... avg=1s min=1s med=1s max=1.44s p(90)=1s p(95)=1s
iterations..... 1010 16.639599/s
vus..... 1 min=1 max=20
vus_max..... 20 min=20 max=20

NETWORK
data_received..... 389 kB 6.4 kB/s
data_sent..... 91 kB 1.5 kB/s
```



```

    http_req_failed: ['rate<0.01'],
  },
};

const BASE_URL = 'http://127.0.0.1:8080/api';

export default function () {
  // LISTAR VULNERABILIDADES (Teste de Leitura - GET)

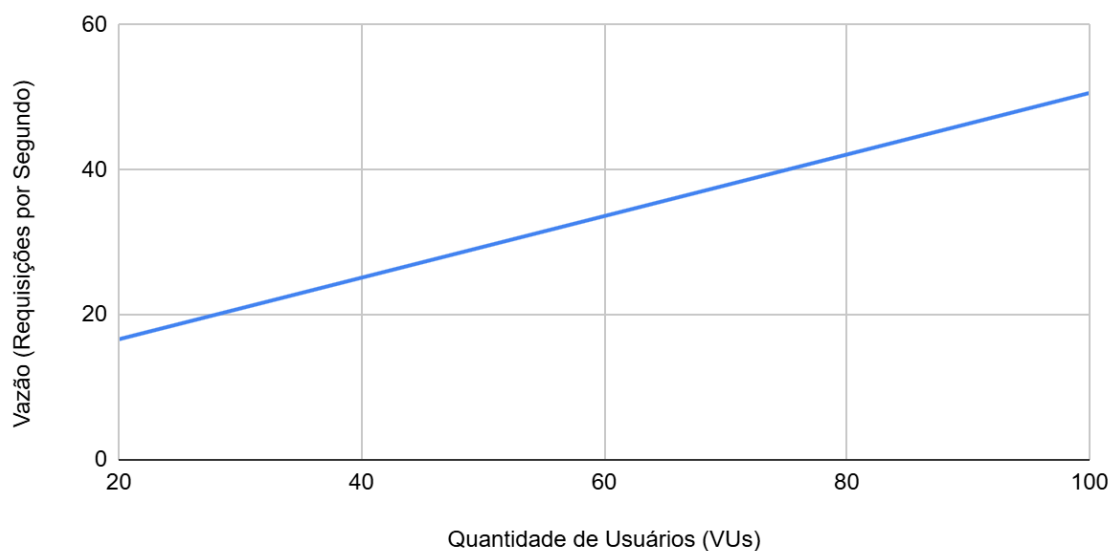
  let res = http.get(`${BASE_URL}/vulnerabilidades`);

  // Verifica se deu certo
  check(res, { 'listagem realizada': (r) => r.status === 200 });

  sleep(1);
}

```

Vazão (Requisições por Segundo) versus Quantidade de Usuários (VUs)



COMPARAÇÃO DE LATÊNCIA ENTRE OPERAÇÃO DE ESCRITA E LEITURA

Latência Média (ms) versus Tipo de Teste

